



Multi-Instance GPU (MIG) Configurations

Optimizing GPU resource allocation for AI workloads in production environments

Why Multi-Instance GPUs?

Modern GPUs like NVIDIA's A100 and H100 deliver immense computational power, but not all AI workloads require the full capacity of these powerful accelerators.

Resource Underutilization

Running small workloads on full GPUs wastes resources and significantly increases cloud costs

Multi-Tenant Requirements

Enterprise environments need to support multiple users and workloads simultaneously



What Is MIG?

Multi-Instance GPU (MIG) is a hardware partitioning feature available on NVIDIA's A100, H100, and L40S GPUs.

1

Hardware Partitioning

Divides a physical GPU into isolated slices that function as independent mini-GPUs

2

Dedicated Resources

Each slice has its own memory and compute resources with no sharing or overlap

3

Complete Isolation

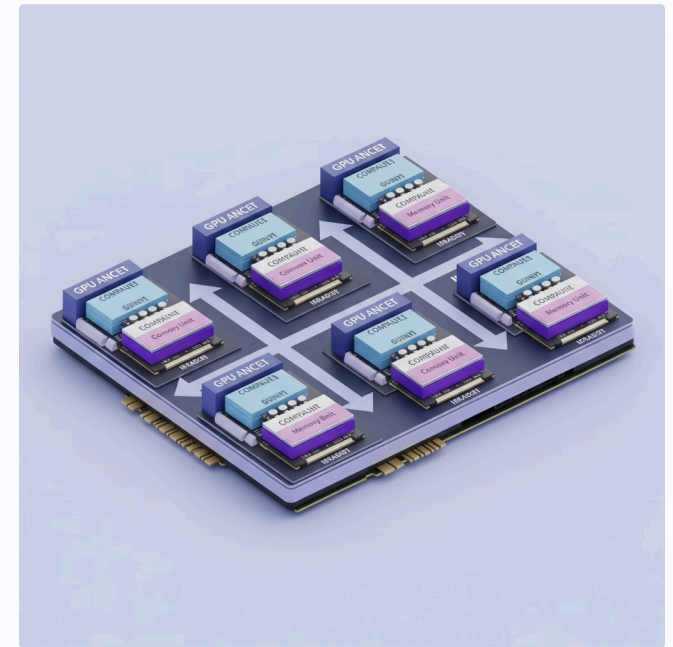
Provides performance predictability with hardware-enforced boundaries between instances

MIG transforms a single powerful GPU into multiple smaller GPUs that can be allocated to different users or workloads independently.

How MIG Works

The MIG architecture divides GPU resources at the hardware level:

- Physical GPU partitioned into **GPU instances**
- Each instance comprises one or more **compute slices** and **memory slices**
- Hardware-enforced isolation prevents cross-instance interference
- Separate CUDA contexts maintain workload independence
- Management through `nvidia-smi mig` command-line tools



Example – A100 MIG Profiles

1g.5gb

1 compute slice

5 GB memory

~1/7 of full GPU

2g.10gb

2 compute slices

10 GB memory

~2/7 of full GPU

3g.20gb

3 compute slices

20 GB memory

~3/7 of full GPU

7g.40gb

7 compute slices

40 GB memory

Full GPU

The A100 GPU can be configured in multiple ways to meet diverse workload requirements. For example, you could create:

- $7 \times 1\text{g.5gb}$ instances for small inference jobs
- $3 \times 2\text{g.10gb} + 1 \times 1\text{g.5gb}$ for mixed workloads
- $2 \times 3\text{g.20gb} + 1 \times 1\text{g.5gb}$ for balanced computing needs

Benefits of MIG



Cost Efficiency

Share expensive GPU resources across teams, reducing the need for additional hardware purchases



Multi-tenancy

Run multiple isolated workloads with guaranteed performance and QoS for each user



Higher Utilization

Improve GPU utilization rates from typical 15-30% to 80%+ through appropriate sizing



Fault Isolation

Prevent failures in one instance from affecting other workloads on the same physical GPU



MIG Use Cases

Inference Microservices

Deploy multiple inference endpoints for chatbots, recommendation engines, and image processing services on a single GPU

Small-Scale Training

Enable academic labs and startups to run multiple parallel experiments with limited GPU resources

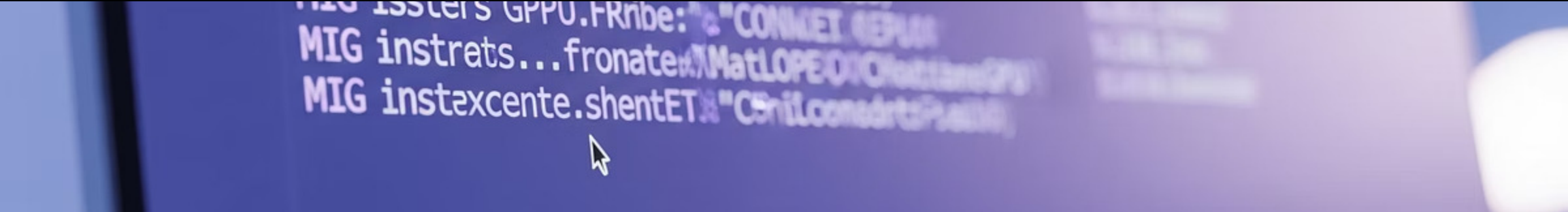
Multi-Tenant GPU Clusters

Create shared infrastructure for data science teams with guaranteed resource isolation

AI SaaS Applications

Optimize cost structure for AI-powered software services with varying workload demands





Enabling MIG

Setting up MIG involves a simple CLI workflow that creates isolated logical GPUs:

```
# Enable MIG mode
sudo nvidia-smi -mig 1

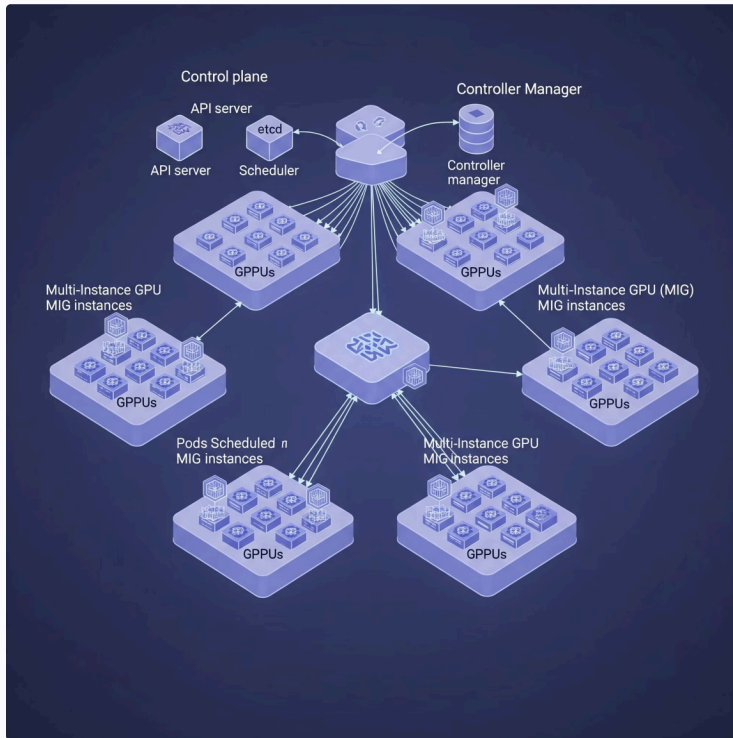
# Create GPU instances (example: two 19% slices)
sudo nvidia-smi mig -cgi 19,19 -C

# List MIG devices
nvidia-smi -L
```

- ❑ After enabling MIG mode, the GPU requires a reboot to apply the changes. Different MIG profiles can be created to match your specific workload requirements.

Once configured, the MIG instances appear as separate GPUs to the operating system and container platforms, enabling fine-grained resource allocation.

MIG + Kubernetes



NVIDIA GPU Operator seamlessly integrates MIG with Kubernetes orchestration:

- Kubernetes scheduler views MIG instances as **distinct GPUs**
- Enables **fine-grained job placement** across GPU resources
- Supports declarative configuration of MIG profiles via CRDs
- Automates MIG instance lifecycle management
- Integrates with resource quotas and namespace isolation
- Compatible with Slurm and other HPC schedulers

This integration is critical for building scalable MLOps pipelines that efficiently utilize GPU resources across teams.

Limitations of MIG



Limited Hardware Support

Only available on Ampere and Hopper architecture GPUs (A100, H100, L40S)



No NVLink Between Instances

MIG instances on the same GPU cannot communicate via high-speed NVLink



Not for Large Models

Unsuitable for large training jobs that require the full GPU memory and compute capacity

Additional considerations:

- Once partitioned, GPU requires reconfiguration to change the MIG profile
- Potential resource waste if workloads don't fully utilize their allocated slice
- More complex monitoring and management requirements
- Limited support in older CUDA applications and frameworks



Key Takeaways

Resource Optimization

MIG transforms a single GPU into multiple isolated mini-GPUs, dramatically improving utilization and cost efficiency

Enterprise Ready

Essential for multi-tenant environments and cloud providers looking to maximize GPU infrastructure ROI

Kubernetes Integration

Works seamlessly with container orchestration for scalable, efficient AI infrastructure

Next steps: Evaluate your current GPU utilization patterns and identify workloads that could benefit from MIG to reduce infrastructure costs.